Data Analysis Project
Final Handin
(Group 05)

Simon Zeugin, Max Reinert, and Alexander Marggraf

University of Basel
Databases (CS246) course
Autumn Semester 2025

1 Introduction

The goal of our project was to identify interesting datasets, create a consistent schema that includes all data sources, clean and integrate the data into a single database, and analyze the data to gain meaningful insights. Throughout this process, we aimed not only to uncover interesting facts about our chosen datasets and the real-world circumstances they model, but also to develop knowledge and skills in finding, cleaning, and integrating real-world, messy data into one coherent database.

The datasets we selected consisted of train and weather data from Switzerland and Germany, which we obtained from the internet. [9][7][6][3] The German train data was provided by Deutsche Bahn and consists of a collection of `.parquet` files which list all recorded train stops of trains operated by Deutsche Bahn in Germany. The collection was already processed such that there is one file per month, which records, among other things, where and when each train stopped and whether it was delayed or canceled. An ER diagram can be seen in Figure 1.

The German weather data was provided by the Deutsche Wetterdienst and consists of a complex collection of text files which are structured similarly to CSV files. These files are separated by measurement type, including temperature, precipitation, pressure, and wind, as well as by historical data up to 2025 and recent data. The files themselves contain the corresponding measurements, the time at which each measurement was taken, and the station at which it was recorded. Additionally, the Deutsche Wetterdienst provided metadata for the weather stations, which offers insight into their individual locations. An ER diagram can be seen in Figure 2.

The Swiss train data was provided by SBB and consists of a collection of large CSV files which list all recorded train stops of trains operated by SBB on a monthly basis. This includes, among other information, the locations at which trains stopped, their exact arrival and departure times, and whether the trains were canceled. An ER diagram can be seen in Figure 3.

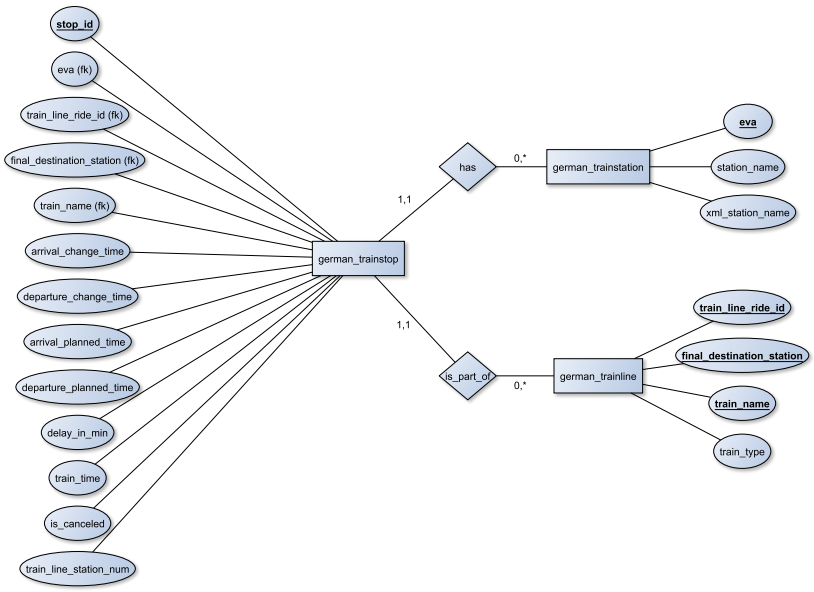


Fig. 1: ER Diagram of the german train dataset

The Swiss weather data was provided by MeteoSchweiz and consists of a collection of large CSV files which list all measured meteorological variables, such as temperature or precipitation, when these measurements were taken, and which station recorded them. Additionally, we obtained a list of all Swiss weather stations recorded by MeteoSchweiz, which can be found here: `https://data.geo.admin.ch/ch.meteoschweiz.ogd-smn/ogd-smn_meta_stations.csv`. An ER diagram can be seen in Figure 4.

Our goal was to determine the delays and cancellation rates of trains in both countries. To achieve this, we considered only data ranging from the beginning of September 2024 to the end of August 2025. Furthermore, we divided the data into two groups, one for each country, and analyzed delays per year, per month, per weather condition, and per canton or municipality. Additionally, we calculated the percentage of train cancellations per canton or municipality and identified which locations experience higher traffic on average. [8]

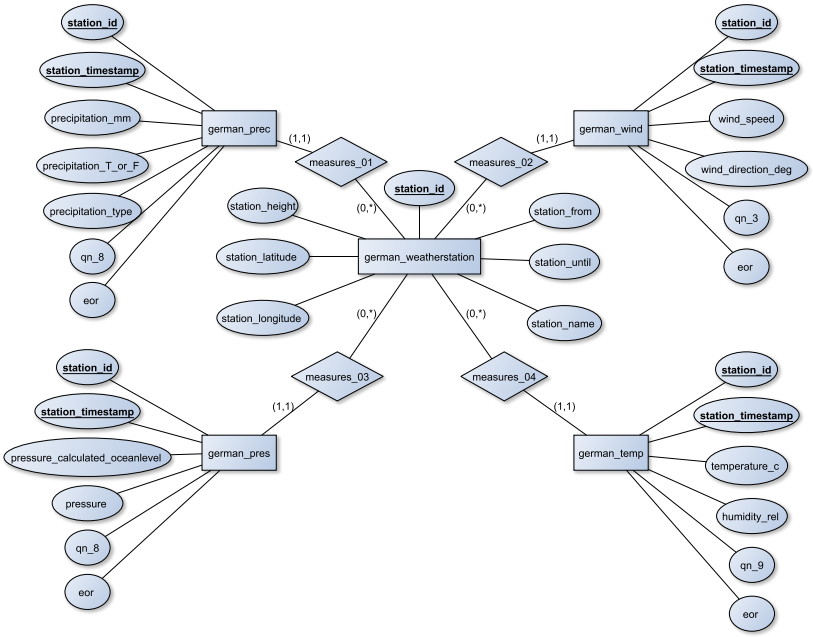


Fig. 2: ER Diagram of the german weather dataset

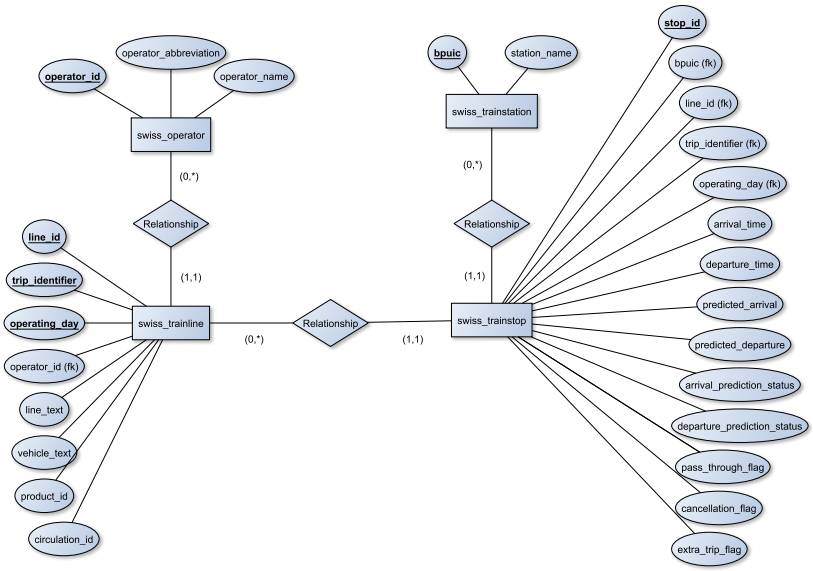


Fig. 3: ER Diagram of the swiss train dataset

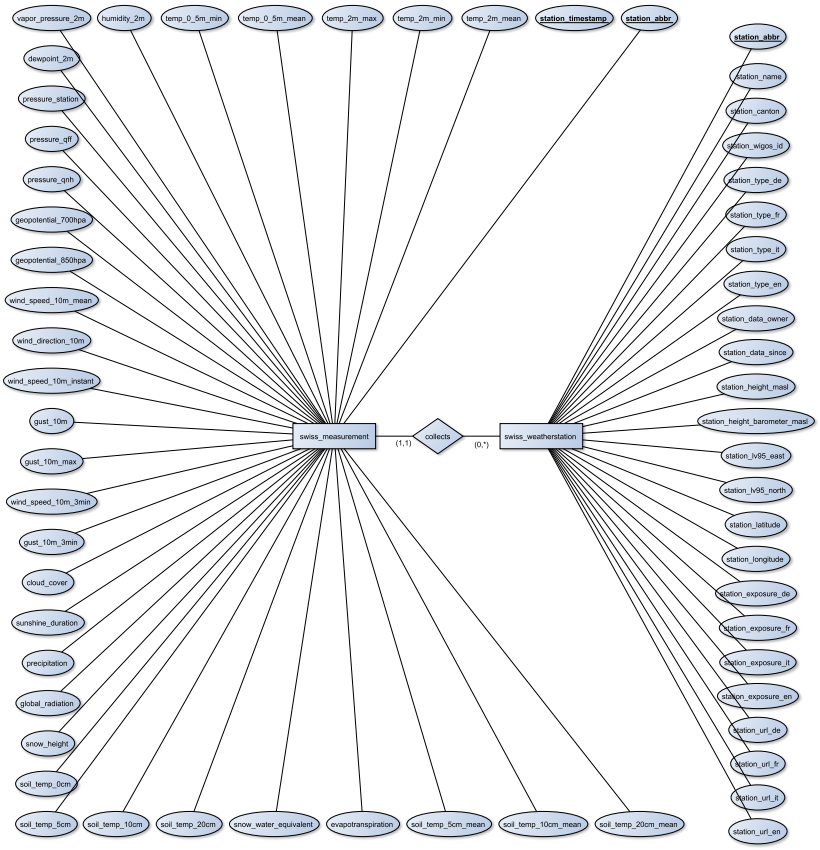


Fig. 4: ER Diagram of the swiss weather dataset

2 Integration

To integrate the data, we first dissected all four datasets and created corresponding ER diagrams. Following that, we attempted to create one large coherent schema which includes all data, but we ended up mostly leaving the datasets unconnected.

For the German train data, we had one `.parquet` file [1] for every month, which we turned into three tables. The `german_trainstation` table holds mostly just the station name but is important for classifying train events later. The `german_trainstop` table holds all information about train events, such as arrival and departure time, where the stop occurred, the delay in minutes, and information about cancellations. The `german_trainline` table holds information about the line under which the train operated, as well as the trains name, type, and its final destination.

The German weather data consisted of many files which were separated by type of measurement and whether the data was historic or recent. For each measurement, we created a table called `german_<measurement_name>` which holds the value of the measurement, additional information, the time the measurement was taken, and the weather station at which it was recorded. The `german_weatherstation` table holds information about the station name as well as its latitude and longitude.

For the Swiss train dataset, we had one set of large CSV files which we turned into four tables. The `swiss_trainstation` table holds mostly just the station name but is important for classifying train events later. The `swiss_trainstop` table holds information about all train stops that occurred, including arrival and departure time, location, and information about cancellations. The `swiss_trainline` table holds information about the train line which operated the train. It contains information about the train being used and the line name. It is important to note that we misinterpreted the dataset in this regard. Instead of just storing line information, we assumed lines differ from day to day, which resulted in a very convoluted primary key consisting of the line ID, a trip identifier, and the operating day. The `swiss_operator` table holds information about the company, such as SBB or ÖBB, under which a train completed its journey.

For the Swiss weather data, we had one set of CSV files which we implemented largely without modification as the table `swiss_measurement`. This table contains information about meteorological measurements, the date of the measurement, and which station recorded the measurement. Additionally, we created a table based on a CSV file of all Swiss weather stations called `swiss_weatherstations`. This table contains extensive data on the different weather stations, but we only use the latitude and longitude for each station.

The integrated schema, which holds all the data, contains all the aforementioned tables as well as one additional table called `trainstation_weatherstation`. This table holds information about which weather station is closest to each train station and also indicates whether the train station or weather station is located in Switzerland or Germany. This is useful for the final analysis. The full ER diagram of the integrated schema can be found in Figure 5, and the logical schema is listed below. Note that the table `trainstation_weatherstation` does not reference the actual tables and merely acts as a lookup table.

swiss_weatherstation(station_abbr, station_name, station_canton, station_wigos_id, station_type_de, station_type_fr, station_type_it, station_type_en, station_data_owner, station_data_since, station_height_masl, station_height_barometer_masl, station_lv95_east, station_lv95_north, station_latitude, station_longitude, station_exposure_de, station_exposure_fr, station_exposure_it, station_exposure_en, station_url_de, station_url_fr, station_url_it, station_url_en)

swiss_measurement(station_abbr, station_timestamp, temp_2m_mean, temp_2m_min, temp_2m_max, temp_0_5m_mean, temp_0_5m_min, humidity_2m, vapor_pressure_2m, dewpoint_2m, pressure_station, pressure_qff, pressure_qnh, geopotential_700hpa, geopotential_850hpa, wind_speed_10m_mean, wind_direction_10m, wind_speed_10m_instant, gust_10m, gust_10m_max, wind_speed_10m_3min, gust_10m_3min, cloud_cover, sunshine_duration, precipitation, global_radiation, snow_height, soil_temp_0cm, soil_temp_5cm, soil_temp_10cm, soil_temp_20cm, snow_water_equivalent, evapotranspiration, soil_temp_5cm_mean, soil_temp_10cm_mean, soil_temp_20cm_mean)

german_weatherstation(station_id, station_height, station_latitude, station_longitude, station_from, station_until, station_name)

german_temp(station_id, station_timestamp, qn_9, temperature_c, humidity_rel, eor)

german_prec(station_id, station_timestamp, qn_8, precipitation_mm, precipitation_T_or_F, precipitation_type, eor)

german_pres(station_id, station_timestamp, qn_8, pressure_calculated_oceanlevel, pressure, eor)

german_wind(station_id, station_timestamp, qn_3, wind_speed, wind_direction_deg, eor)

swiss_trainstation(bpuic, station_name)

swiss_operator(operator_id, operator_abbreviation, operator_name)

swiss_trainline(line_id, trip_identifier, operating_day, operator_id, line_text, vehicle_text, product_id, circulation_id)

swiss_trainstop(stop_id, bpuic, line_id, trip_identifier, operating_day, arrival_time, departure_time, predicted_arrival, predicted_departure, arrival_prediction_status, departure_prediction_status, pass_through_flag, cancellation_flag, extra_trip_flag)

german_trainstation(eva, station_name, xml_station_name)

german_trainline(train_line_ride_id, final_destination_station, train_name, train_type)

german_trainstop(stop_id, eva, train_line_ride_id, final_destination_station, train_name, arrival_change_time, departure_change_time, arrival_planned_time, departure_planned_time, delay_in_min, train_time, is_canceled, train_line_station_num)

trainstation_weatherstation(trainstation, trainstation_in_ch, weatherstation, weatherstation_in_ch)

To integrate the schema, we used the same approach for each dataset. This

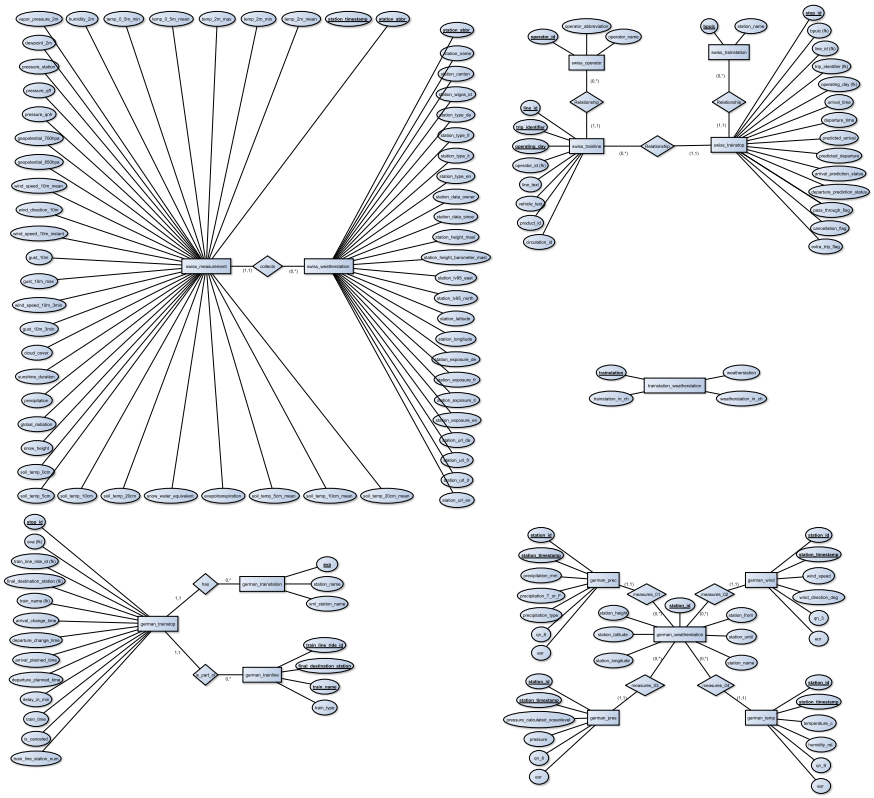


Fig. 5: ER Diagram of the final database

included loading one source file into a pandas DataFrame, loading any CSV files which needed to be extended (e.g., the `swiss_trainstation` CSV), discarding all rows which were missing a necessary column value such as the primary key, creating DataFrames for each table related to that CSV, filling the DataFrames with data from the source file or extending them using concatenation and deduplication, performing a final deduplication step to catch any remaining duplicates, and storing all necessary DataFrames either in a new CSV file or in the CSV from which the data was loaded. After one file was processed, we repeated this procedure until all files had been iterated over. All resulting CSV files could then easily be loaded into the database using the `COPY` command. [11] This was significantly faster than performing inserts for each individual data point and

took at most 10 to 15 minutes. The processes differed slightly for each dataset, which we discuss in the following sections.

For the German train data, we first needed to convert the monthly source files from Parquet to CSV to avoid pandas DataFrames containing columns with the `object` data type.

For the German weather data, we had a large number of files which were separated into folders by temperature, precipitation, pressure, and wind speed, as well as by historical data ranging up to 2025 and recent data ranging from the beginning of 2025 until now. Each folder contained one subfolder per weather station, holding both the measurement data and the station data. This significantly complicated the process, as we had to iterate over all possible file locations to locate all relevant data. Furthermore, all data was stored in text files formatted similarly to CSV files but containing additional whitespace. These whitespaces had to be removed before processing the data. Additionally, we needed to filter the data to the correct time span, as data from outside our specified period was present. To clean and normalize the data, cast some columns from float to int or from int to boolean to correctly represent the measurements, set impossible data points to NA, and format dates to match the PostgreSQL date and timestamp format. For the station data, we additionally retained only the most recently recorded metadata to avoid duplicated weather stations.

For the Swiss train data, we used an artificial ID in the form of an increasing integer. This required us to load and store the current ID value in a JSON file for each CSV processed. Additionally, we filtered out all non-train events, as they did not fit our specified analysis goals and would have significantly increased the dataset size. Originally, the raw SBB data amounted to approximately 170 GB. We also formatted dates to ensure compatibility with the PostgreSQL date and timestamp format.

For the Swiss weather data, we similarly formatted dates to the PostgreSQL date and timestamp format and filtered the data to the correct time span, as the dataset also contained measurements outside our specified period.

The `trainstation_weatherstation` table was populated only after all other tables had already been filled. We loaded all train stations and weather stations into pandas DataFrames to determine which weather station was closest to each train station. Fortunately, the weather stations in both the German and Swiss datasets were documented with latitude and longitude. Therefore, we only needed to determine the coordinates of the train stations. To achieve this, we used the Nominatim interface [5] of `geopy` to locate the train stations. We discarded all stations that were not located in Germany or Switzerland, as our analysis focused exclusively on these countries. This approach worked well in most cases; however, for approximately 40 train stations, the locations had to be

determined manually. Afterwards, we constructed a k-d tree [10] using `scipy`, a data structure designed to organize points in k dimensions, which in our case was two. Using built-in functions, we then identified the closest weather station for each train station and stored the resulting data in a CSV file, which was again inserted into the database using the `COPY` command. [8]

3 API

For the API, FastAPI[2] was used to implement a REST interface for accessing the precipitation data stored in the german_prec table. The API provides basic reading (GET), adding (POST), modifying (PUT) and deleting (DELETE) functionality. The service connects directly to the database using psycopg2.
The API can be started locally using:

```
python -m uvicorn main:app --reload
```

Alternatively, it can be started via the provided `run_fastapi.bat` file.

After startup, the interactive API documentation is available at:

```
http://127.0.0.1:8000/docs
```

3.1 Data Model

All requests that insert or update data use the same JSON structure, defined using a Pydantic model. Each record is uniquely identified by a composite primary key consisting of the station_id and the station_timestamp. All remaining attributes are optional.

Example JSON object:

```
{
  "station_id": 164,
  "station_timestamp": "2024-09-01T00:00:00",
  "qn_8": 3,
  "precipitation_mm": 0,
  "precipitation_t_or_f": false,
  "precipitation_type": 0,
  "eor": "eor"
}
```

3.2 GET: Read Precipitation Data

Endpoint:

```
GET /precipitation/{station_id}
```

This endpoint returns up to 100 precipitation records for a given weather station. The limit is applied to prevent excessive data transfer and ensure acceptable response times.

Example request:

```
GET /precipitation/164
```

Example response:


```
[
  {
    "station_id": 164,
    "station_timestamp": "2024-09-01T00:00:00",
    "qn_8": 3,
    "precipitation_mm": 0,
    "precipitation_t_or_f": false,
    "precipitation_type": 0,
    "eor": "eor"
  }
]
```


3.3 POST: Insert New Data

Endpoint:

```
POST /precipitation
```

This endpoint inserts a new precipitation record into the database.

Example request body:


```
{
  "station_id": 1,
  "station_timestamp": "2024-09-01T00:00:00",
  "qn_8": 3,
  "precipitation_mm": 0,
  "precipitation_t_or_f": false,
  "precipitation_type": 0,
  "eor": "eor"
}
```


Example response:


```
{
  "message": "new tuple created"
}
```

3.4 PUT: Update Existing Data

Endpoint:

`PUT /precipitation/{station_id}/{station_timestamp}`

This endpoint updates an existing precipitation record identified by station ID and timestamp. The primary key itself cannot be modified.

Example request:

`PUT /precipitation/1/2024-09-01T00:00:00`

Example request body:

```
{
  "station_id": 1,
  "station_timestamp": "2024-09-01T00:00:00",
  "qn_8": 3,
  "precipitation_mm": 5,
  "precipitation_t_or_f": true,
  "precipitation_type": 1,
  "eor": "eor"
}
```

Example response:

```
{
  "message": "updated"
}
```

3.5 DELETE: Remove Data

Endpoint:

`DELETE /precipitation/{station_id}/{station_timestamp}`

This endpoint deletes a precipitation record using station ID and timestamp as identifiers. In the following example, the artificially created entry from the POST example is removed.

Example request:

`DELETE /precipitation/1/2024-09-01T00:00:00`

Example response:

```
{
  "message": "deleted"
}
```

[8]

4 Analysis

For the analysis, we wanted to see the differences in train delays in Switzerland and Germany and how much they are effected by the weather in those countries. First, we examined the average train delay in each country. We then analyzed how weather conditions affected the mean delay. We also did a statistical analysis to properly interpret these results. The next step was to further split the data to obtain more detailed result. So we checked how a specific weather impacted the average delay. Since our datasets did not explicitly indicate the specific weather present at a given time, we had to take another approach for this. Both datasets, the Swiss weather dataset and the German data set measured precipitation in mm which is an indicator for the weather. Therefore, the table shown in Figure 6 was used to map the precipitation levels to weather categories. We also looked

weather type	dry	light rain	rain	heavy rain	snow
precipitation	0	0 - 5	5 - 10	> 10	> 0 with temp < 0

Fig. 6: Table used to map precipitation to weather

at the impact of each precipitation on the average delay using precipitation intervals of 5 mm. Since we already took a look oon the impact of the weather, we also wanted to examine whether there is a correlation between train delays and the month of the year. Our datasets covered the period from April 2024 to September 2025, so we looked for all the months in that time span. We also went a step further and checked the delays and cancellations for the individual Cantons and Counties with respect to the total train stops.

Queries used for the analysis

In order to get proper results and functioning queries we had to take a look at the datasets themselves. Here we noticed that the Swiss train data set had train stations from different countries and from Germany in. In the analysis, this would have led to some problems since some german stations would appear twice with different delays. In order to fix this issue, we modified the final database such that we can check if the train station comes from the Swiss or German data set. With this change, we could get more meaningful results. For the average delay and cancellations per Canton and County, we used geopy to map each train station to the corresponding Canton and County.
In order to get the data from the database, we always divided the query into four smaller queries. One for each dataset, the German train and weather data and one for the Swiss train and weather data. For the analysis of the average delay per Canton and County, we moved some of the logic and calculations into the query since they were not too complex. Here we also had to keep in mind to write a query which checks the German train stops in the Swiss train data

set and add them to the German data. In order to keep the queries a bit more simple, we also added a filter to it to the query more readable.
For the average delay per country, we kept the logic out of the queries since this would have led to much greater running time of our code. So here we kept the queries as simple as possible and did all the calculations with the data in the code itself.
Taking a closer look at the results we also noticed that some delays were negative and some were absurdly large. Here we had to adjust a train with negative delay, which means that it was arriving too early to a delay of 0 and the large outliers had the problem that the time stamps got mixed up so we subtracted 1440 minutes form them which led to accurate results.

Visualisation methods used for the analysis

For the visualisation of the results form the average delay per Canton and County, we used the package choropleth maps. With this, we could easily display the results in a clean and readable format. This meant that we also had to get some GeoJSON files for Switzerland, Germany, Cantons and Counties. For the display of the impact of the weather and the precipitation on the average delay and how the average delay changes per month, we used bar charts. To create the charts, we used matplotlib and numpy. To be able to make calculations with such big data, we also used dataframes from pandas. With these approaches, we were able to get a clean and readable analysis with readable and efficient code.

Results of the analysis

In this section, we now take a closer look at the final results of our analysis of the average delay for each country, Cantons and Counties and how much the weather effected the average delay in Switzerland and Germany.

Delay per Country and the effect of the weather

The first thing we chacked is the delay per county in order to get a solid baseline and to check if the data is reasonable. Here we calculated the average delay in minutes, the median delay, the standard deviation of delays and the actual count of train stops. Figure 7 shows our results, we rounded to the second decimal place and the delay is calculated in minutes. Looking at the results of this

country	mean delay	median delay	sd. of delays	count of train stops
Switzerland	1.06	0.63	2.94	40137204
Germany	4.52	2.00	9.96	3391881

Fig. 7: This table shows the mean delay per country

figure, we notice that first of all, the Swiss data set has much more train stops

and is therefore much more detailed and accurate. We also notice that in Germany, we should expect that any train has 3.46 minutes more delay than a train in Switzerland. This means that the trains in Switzerland are way more on time than in Germany.

Since we have a solid baseline now, we now checked the impact of the weather on the average delay. For this we said that normal weather means that the precipitation is smaller than 5 and bad weather means that the precipitation is greater or equal to 5. To get the actual effect now, we subtract the average delay for normal weather from the average delay from bad weather. With this, we get the actual effect on the average delay. The results of this calculation are shown in Figure 8, the delays are shown in minutes and rounded to the second decimal place. In this figure we see that in Switzerland, a train is expected to

country	delay normal weather	delay bad weather	effect of the weather
Switzerland	1.06	1.33	0.27
Germany	4.52	5.23	1.01

Fig. 8: This figure shows the effect of the weather on the average delay

have a delay of 0.27 minutes more with bad weather than with normal weather. For germany, we should expect that the delay increases by 1.01 minutes if the weather is bad. This shows that the impact of the weather on the German train net is much greater than on the Swiss train net. This also means that even with bad weather, the Swiss trains are much more on time than German trains for each weather.

The datasets differ very much in their size and the actual count of delays per normal weather and bad weather also differ very much. In order to make more accurate conclusions, we also ran a Mann-Whitney-test [4] for Germany and Switzerland for the normal and bad weather. Figure 9 shows the results of the Mann-Whitney-test. Here, a larger value of the U-statistic indicates a stronger

country	U-statistic	p-value	$n_{bad_weather}$	$n_{normal_weather}$
Switzerland	2.55×10^{12}	0.0	116921	40020283
Germany	9.96×10^{9}	3.6×10^{-21}	5486	3386395

Fig. 9: This table shows the results for the Mann-Whitney-test

tendency for delays during bad weather to be higher than during normal weather. For both countries, we see that the U-statistic value is really large so there is a strong tendency that bad weather increases the delays. The p-value is an indicator if the hypothesis that bad weather increases the delay is true or not. The

closer this is to 0, the higher is the chance that the delay is indeed affected by bad weather. For both countries the p-value is really close to zero which means that we can be sure that the delays are increased by bad weather. We also see that the sample size for bad weather is much smaller than for good weather which means that interpreting the results solely based on total counts should be done with caution.

At the end we also made a regression model for our data. For the regression model, the data from Switzerland was our baseline. We specified the following regression model to analyze the effect of weather and country on train delays:

$$\begin{aligned}\text{delay}_i = \beta_0 + \beta_1 \times \text{Precipitation}_i + \beta_2 \times \text{Temperature}_i + \beta_3 \times \text{WindSpeed}_i \\ + \beta_4 \times \mathbb{1}(\text{DE}_i) + \beta_5 \times \Big(\text{Precipitation}_i \times \mathbb{1}(\text{DE}_i)\Big) + \varepsilon_i\end{aligned}$$

We created the regression model based on the formula, and we got the results for the variables, shown in Figure 10. The coefficient indicates how much the

variable	coefficient	standard error	t-value	p-value
intercept	0.995	0.001	896.866	0.000
precipitation	0.085	0.001	124.823	0.000
temperature	0.001	7.844	15.726	0.000
wind speed	0.008	0.000	45.776	0.000
country DE	3.476	0.006	630.647	0.000
precipitation: DE	0.159	0.013	12.285	0.000

Fig. 10: Variables of the regression model

average delay changes when the corresponding variable increases by one unit. This indicates that precipitation has the strongest impact on train delays and that the temperature and the wind speed have minimal effects on the actual delay.

Average delay and cancellations per Canton and County with respect to total train stops

We were also interested in the average delay in each Canton and County. This is shown in Figure 11 and Figure 12. In these two maps, we can see that not all counties in Germany are represented in our date. We can also see that in Germany, the delays in some counties were too high, such that we had to use a logarithmic scale for the coloring. We can also see that the highest delay in the Cantons occurs in Basel-Stadt and is around 2 minutes. This is still smaller than the average delay in all counties of Germany. We can also see that in some German counties, the average delays got pretty big. This is due to the fact that some counties had the number of train stop events in the single digits, which

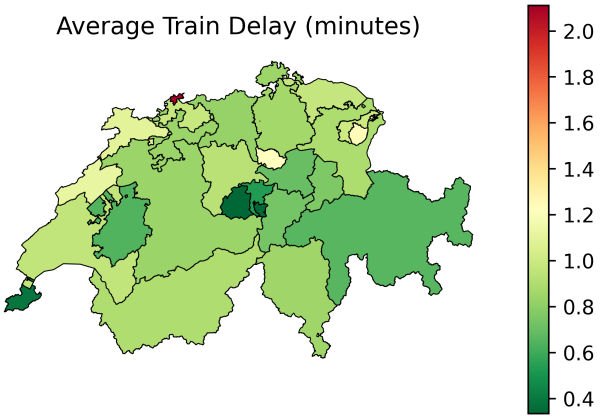


Fig. 11: Average delay per Canton

leads to pretty big delay values. Even with considering this, the average delays in the counties are much larger than in all of the cantons of Switzerland.

We also take a look at the cancellations per canton and counties. The results are shown in Figure 13 and Figure 14. In these figures, we can also see that the actual percentage of trains canceled in the cantons and counties is pretty similar with some small exceptions. Here we can see that most of the trains are fairly reliable but Switzerland seems to be more reliable than Germany. For the cancellations in the canton of Basel-Stadt we also see a higher value than for the other cantons. Since this appears in both graphics, the average delays and the cancellation, there might be a correlation with fact that Basel-Stadt borders Germany and ist a big entry points for trains from Germany. This could lead to the assumption that German trains have an impact on the reliability of the train net in Basel-Stadt.

We also considered looking at total train stops per canton and county in Figure 15 and Figure 16 to find regions with a very small count of trains and to see a how reliable the other graphs are for the cantons and counties. Looking at this total train stops, we can see that most of the data is pretty reliable but there are some regions with way more data than others. We can also see that the total amount of stops in Switzerland is larger than in Germany, this means that the Swiss data is muh more accurate than Germany. Since Germany might be missing a lot of counties with very punctual trains, which would decrease the average delay or very delayed regions which would increase the average delay.

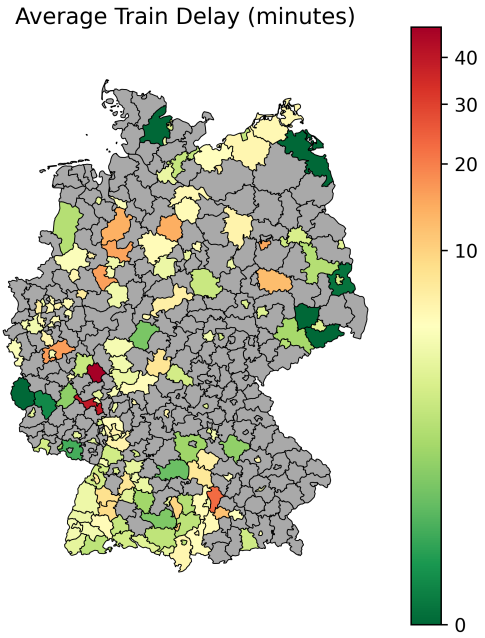


Fig. 12: Average delay per County

Delay per Weather

We also checked the average delay for each weather for Switzerland and Germany. The results of this are shown in Figure 17. This chart shows that the average delay in Switzerland is actually pretty much the same for each weather except for snowy weather. Here the trains have a delay which is about twice as large as for the other weather. In Switzerland, the trains also exhibit relatively similar average delays across weather categories but for Germany, the biggest delay seems to occur for heavy rain. As expected, the trains have the smallest average delay for good and dry weather. An interesting fact is that the smallest average delay for German trains is still almost twice as large as the average delay for snowy weather in Switzerland.

Delay per Precipitation

After taking a look on the weather based on precipitation, we also wanted to examine finer precipitation intervals to get a more detailed look on the effect of the precipitation. Figure 18 shows the results for this analysis. In this chart, we see that the German data is actually missing the data for larger precipitation

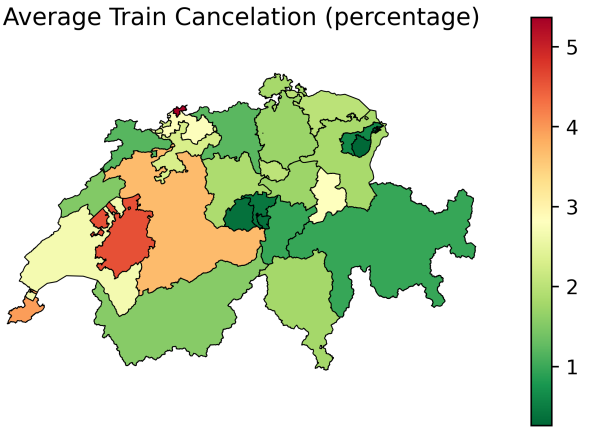


Fig. 13: Cancellations per Canton

measurements. For Switzerland, we can see that the effect of the precipitation on the average delay seems to e pretty small. Since for even larger values of precipitation, the average delays are pretty similar to the smaller values of precipitation. But we can see that there are more individual delays which are much larger if the precipitation is much higher. For Germany, we can see a much bigger impact of the precipitation on the average delay. This could show that the German train net is much more effected by precipitation than the Swiss tain net. This also leads to the assumption, that the Swiss train net is way more robust than the German train net.

Delay per Month

Our final analysis is how the average delay changes over the months. We expected to see a higher average delay in the colder winter months than in the warmer summer months with better weather. In Figure 19 we can again see that the Swiss trains are much more on time than the German trains. We can also see that the Swiss trains have the highest delay in November, which is expected, since this month is often colder with more rain and snow. For Germany, we see that the average delay over the months is much more inconsistent. Here the worst month is again November, which is expected for the same reasons as for Switzerland. It is surprising that the trains seem to be the most punctual in January for both Switzerland and Germany, here we also expected to see that the average delay is higher than for the other months. We have to keep in mind that based on our time span, we are missing the summer months with the best weather. Based on the consistency of the average delays of the Swiss trains, we do not expect to see a decrease in the average delay in those months. For Germany,

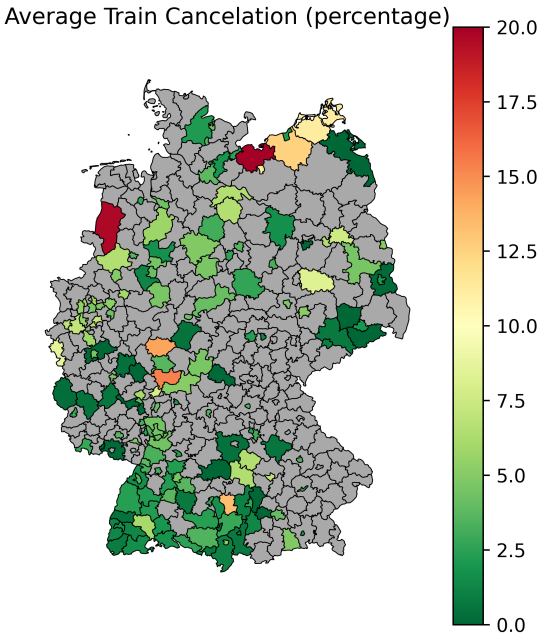


Fig. 14: Cancellations per County

in the summer the average delays might be smaller in the summer since the weather has a much higher impact in Germany and the trains in Germany are much more inconsistent than in Switzerland.

Conclusion of the analysis

At the end, we can say that the Swiss trains are much more punctual and reliable with an average delay of 1.06 minutes than the German trains with an average delay of 4.52 minutes. We can also see that the impact of the weather and the month in Switzerland is pretty small and in Germany, the effect is much more noticeable. This leads to the assumptions that the German train net is much more unreliable and less robust than the Swiss train net. We can also see that the trains in each Canton are way more reliable than in the Counties of Germany. The trains in Switzerland are also less likely to be canceled than the German trains.

We have to keep in mind that the Swiss data is much larger than the German data, which means that the Swiss data is way more accurate than the German

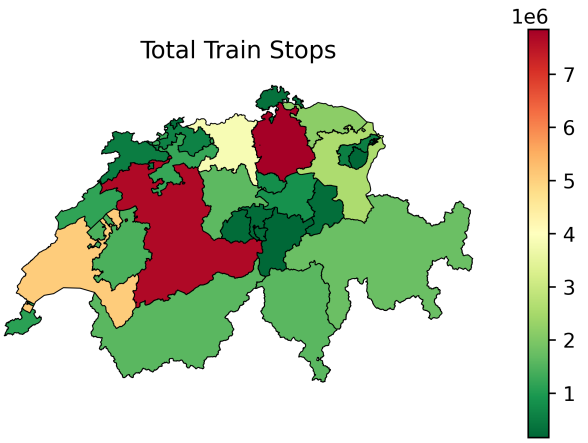


Fig. 15: Total stops per Canton

data. But the German data is still pretty big which means that the conclusion might still be correct but to get better results, we should take a look on larger datasets from Germany. What we also have to consider is that there were some outliers in the datasets which have a delay of over one day, which is most likely due to the fact that there was a mistake in the actual timestamp in the datasets. For these outliers and the trains which arrived earlier than expected, we changed the calculated delay. Here we might have had wrong assumptions, and we might have had an influence on the actual data, and we might have missed some outliers, which are still in the results.

In the end, we can conclude that our results are meaningful and consistent with real-world experience.

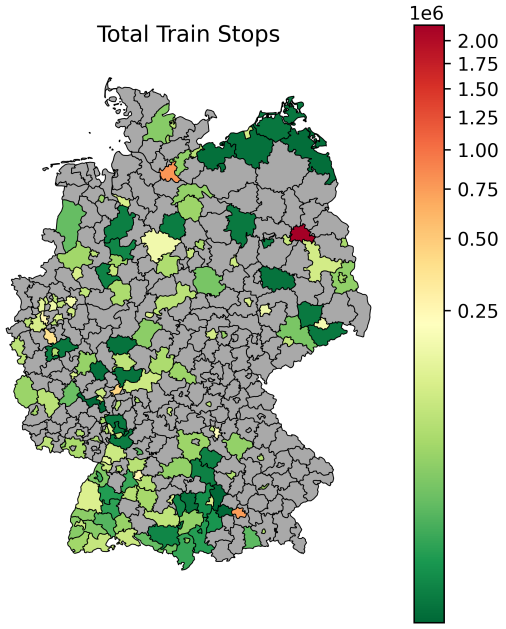


Fig. 16: Total stops per County

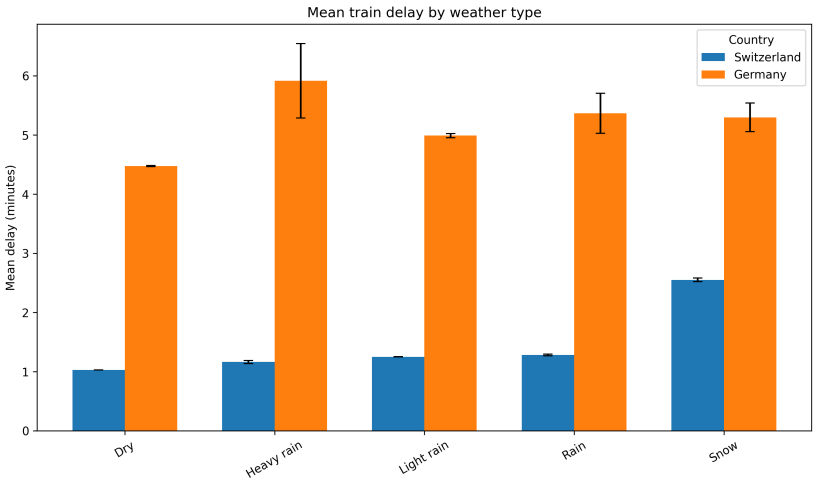


Fig. 17: Average delay per weather

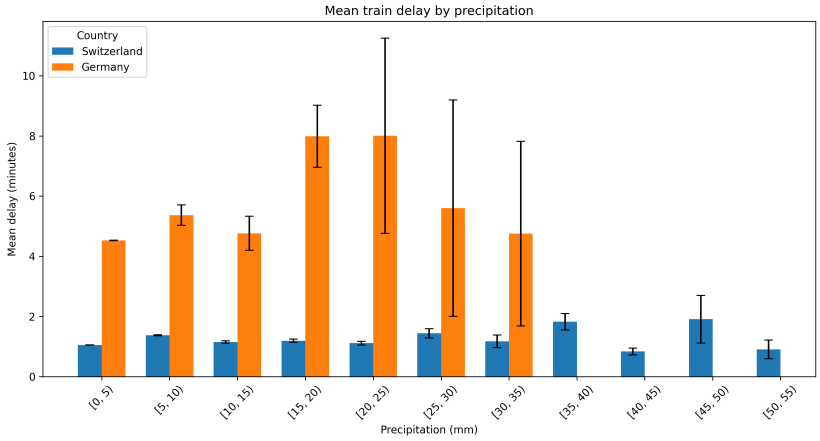


Fig. 18: Average delay per precipitation

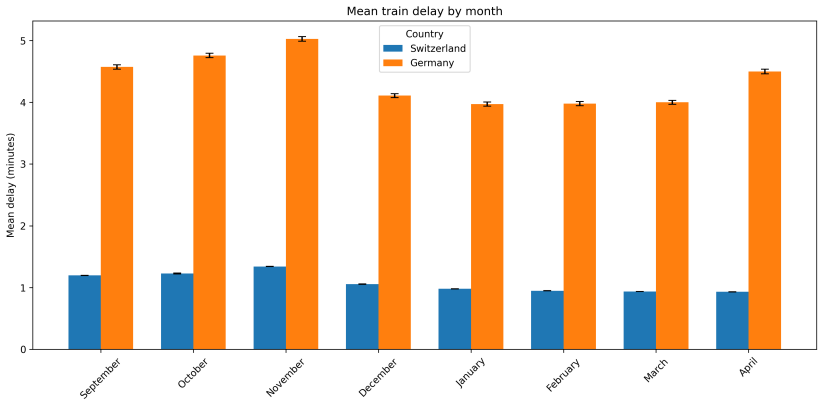


Fig. 19: Average delay per month

References

[1] *Apache Parquet*. https://parquet.apache.org/. 2025.

[2] FastAPI. *FastAPI Documentation*. https://fastapi.tiangolo.com/. 2025.

[3] *Ist-Daten Archiv*. https://archive.opentransportdata.swiss/actual_data_archive.htm. 2025.

[4] H. B. Mann and D. R. Whitney. "On a Test of Whether One of Two Random Variables Is Stochastically Larger Than the Other". In: (1947).

[5] *Nominatim — OpenStreetMap Search Engine*. https://nominatim.org/. 2025.

[6] *Open Data Download Documentation*. https://opendatadocs.meteoswiss.ch/de/general/download. 2025.

[7] *Open Data Portal*. https://opendata.dwd.de/. 2025.

[8] OpenAI. *ChatGPT (GPT-5) language model*. https://chat.openai.com/. Used to assist with spelling, grammar, rephrasing, and technical support for LaTeX notation and formatting in this project. 2026.

[9] piebro. *Deutsche Bahn Data*. https://github.com/piebro/deutsche-bahn-data. 2025.

[10] *scipy.spatial.KDTree — SciPy v1 Reference Guide*. SciPy Community. 2025. URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.KDTree.html.

[11] The PostgreSQL Global Development Group. *SQL COPY — PostgreSQL Documentation*. PostgreSQL Global Development Group. 2025.